

**Payment System Architecture
(Stripe + Webhook + Backend)
Piezo Pico-to-Femtotechnology Sensors Inc.
Version 1.0 — August 2026**

1. Overview

This document explains how a modern commercial payment system works using Stripe as the payment processor. It covers the full lifecycle: from adding items to a cart, creating a checkout session, Stripe communicating with banks, verifying payments, and sending webhook events back to your backend. It also explains Stripe keys, environment configuration, and the backend architecture required to support a production-ready store.

2. Stripe Keys and Environment Configuration

2.1 Types of Stripe Keys

Stripe provides four main keys:

- Publishable Key (pk_live_... or pk_test_...) Used only on the frontend. Identifies your Stripe account but cannot perform sensitive operations.
- Secret Key (sk_live_... or sk_test_...) Used only on the backend. Can create charges, checkout sessions, refunds, subscriptions.
- Webhook Secret (whsec_...) Used only on the backend. Validates that incoming webhook events truly come from Stripe.
- Restricted Keys (optional) Custom keys with limited permissions for microservices.

2.2 Where Keys Are Stored

Keys must never be hard-coded.

Use environment files:

- .env → production keys
- .env.local → development keys
- .env.example → template for other developers

2.3 How Keys Are Imported

Backend (FastAPI):

```
import os
STRIPE_SECRET_KEY = os.getenv("STRIPE_SECRET_KEY")
STRIPE_WEBHOOK_SECRET = os.getenv("STRIPE_WEBHOOK_SECRET")
```

Frontend (Next.js):

```
process.env.NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY
```

3. Store Architecture (Cart → Checkout → Verification)

3.1 Add to Cart

Users select items and add them to a cart stored in:

- Database (cart_items table)
- Or frontend state (temporary)
- Or both (recommended)

Each cart item includes:

- item_id
- item_name
- price_usd
- quantity
- billing (one-time or subscription)

3.2 Create Checkout Session

Backend creates a Stripe Checkout Session:

1. User clicks “Checkout”.
2. Frontend sends cart items to backend.
3. Backend creates a Stripe session:

```
stripe.checkout.Session.create(...)
```

4. Backend returns session.url.
5. Frontend redirects user to Stripe’s secure payment page.

3.3 User Pays Stripe

User enters:

- Card number
- Expiration
- CVV
- Billing address

Stripe encrypts all data and sends it to the bank.

4. Stripe → Bank Communication Flow

4.1 Authorization

Stripe sends the payment request to the user's bank.

Bank checks:

- Card validity
- Available funds
- Fraud detection
- 3D Secure (if required)

4.2 Verification

Bank returns:

- Authorized → Payment approved
- Declined → Insufficient funds, fraud, expired card
- Requires authentication → 3D Secure challenge

4.3 Capture

Stripe finalizes the charge and marks the payment as:

- `payment_intent.succeeded`
- `payment_intent.failed`
- `payment_intent.requires_action`

5. Stripe Webhook → Backend Verification

5.1 Why Webhooks Are Required

Frontend cannot be trusted to confirm payments. Stripe sends a server-to-server message to your backend.

5.2 Webhook Flow

1. Stripe finishes payment.
2. Stripe sends a webhook event to your backend URL: POST /webhook/stripe
3. Backend verifies the event using STRIPE_WEBHOOK_SECRET.
4. Backend updates database:
 - Mark order as paid
 - Activate subscription
 - Send email receipt
5. Backend responds 200 OK.

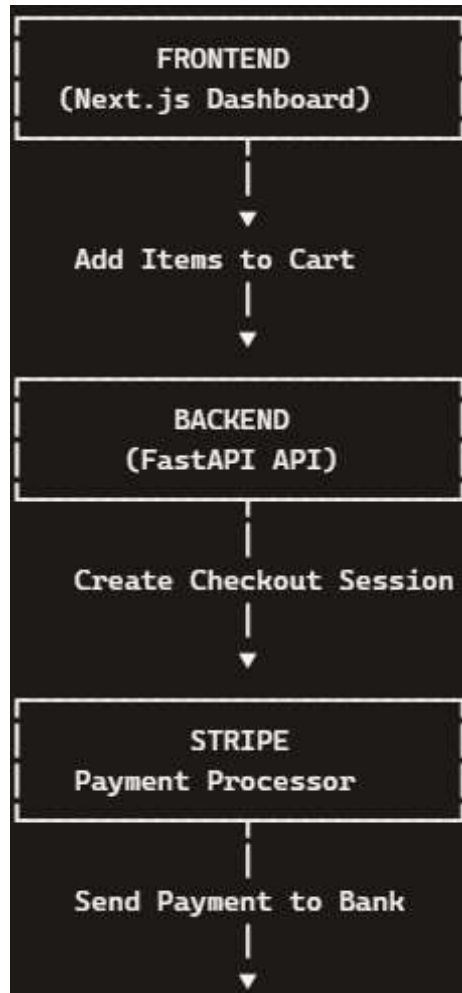
5.3 Common Webhook Events

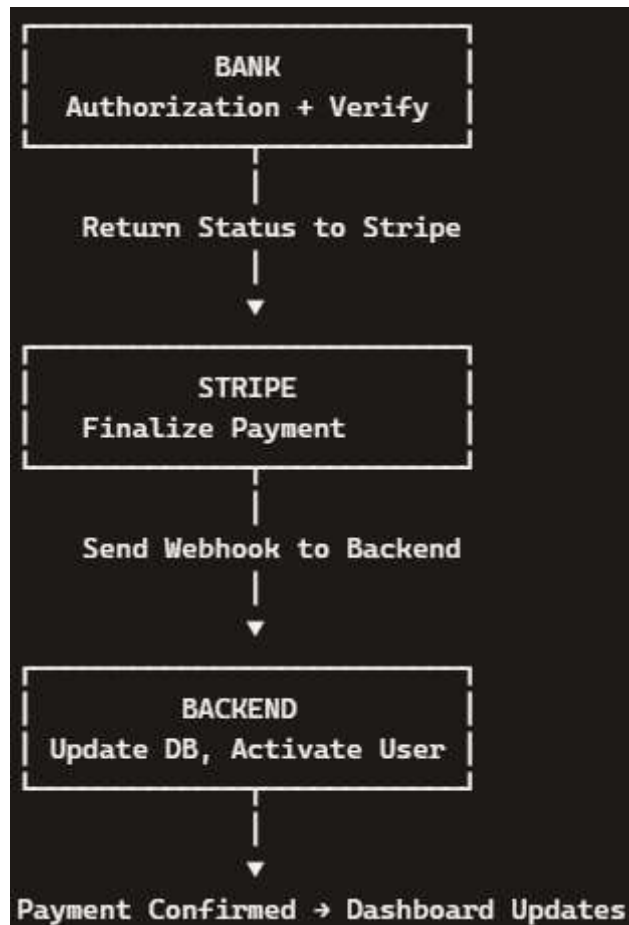
- checkout.session.completed
- payment_intent.succeeded
- payment_intent.payment_failed
- customer.subscription.created
- invoice.paid

6. Full Payment Lifecycle Diagram

Code

See schematic diagram below:





7. Production-Ready Best Practices

- Always verify webhook signatures.
- Never trust frontend payment confirmation.
- Store Stripe keys in environment variables only.
- Use HTTPS for all webhook endpoints.
- Log all payment events for auditing.
- Use idempotency keys for retry-safe operations.
- Use Stripe test mode before going live.

8. Publication & Contact Information

Publisher:

Piezo Pico-to-Femtotechnology Sensors Inc. Edmonton, Alberta, Canada

Contact:

- **Email:** selemaniseif12@yahoo.com or selemaniseif1974@gmail.com
- **Website:** Piezo Pico to Femtotechnology Sensors Inc.
- **Support:** selemaniseif12@yahoo.com
- **Phone:** 780 6044032

Copyright

© 2026 Piezo Pico-to-Femtotechnology Sensors Inc. All rights reserved.

9. Summary

This document provides a complete, professional overview of how Stripe integrates with your backend, how payments are verified, how webhooks work, and how a commercial store architecture is designed. It is suitable for your dashboard documentation, your API engineering course, and your system architecture diagrams.